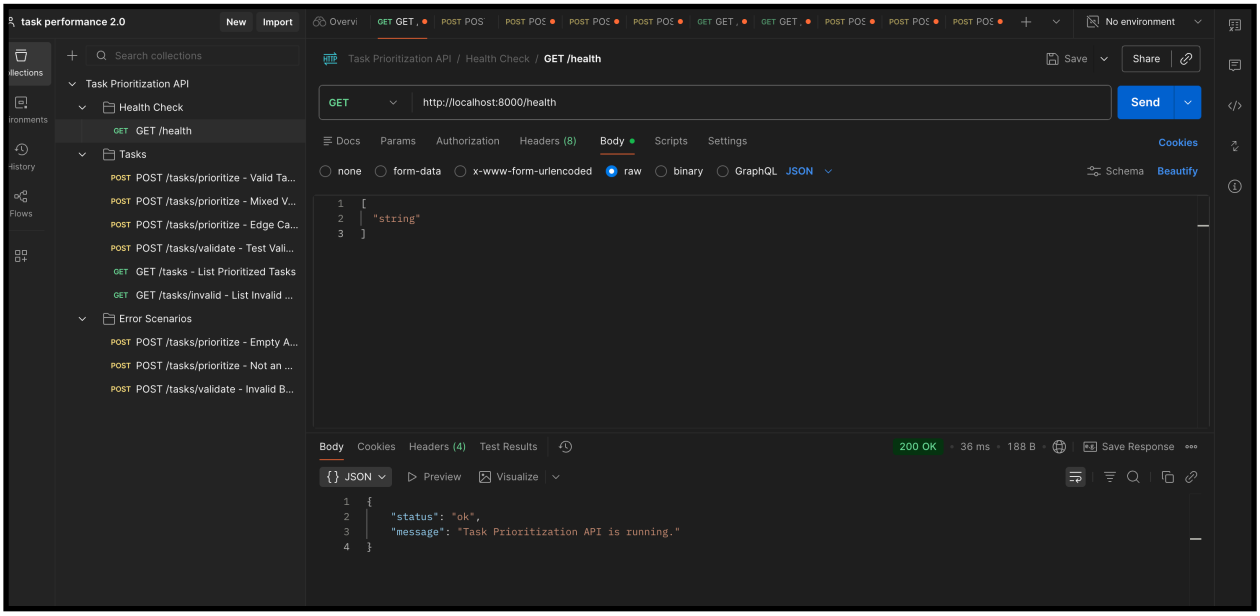
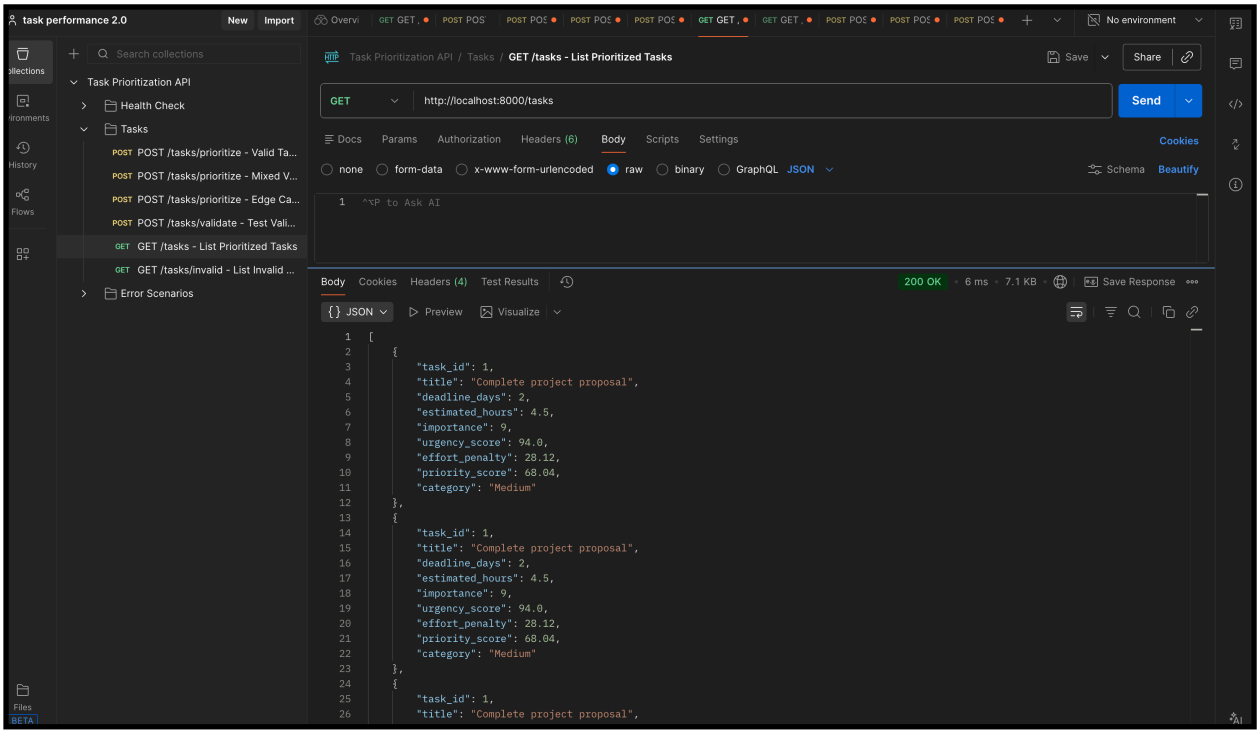
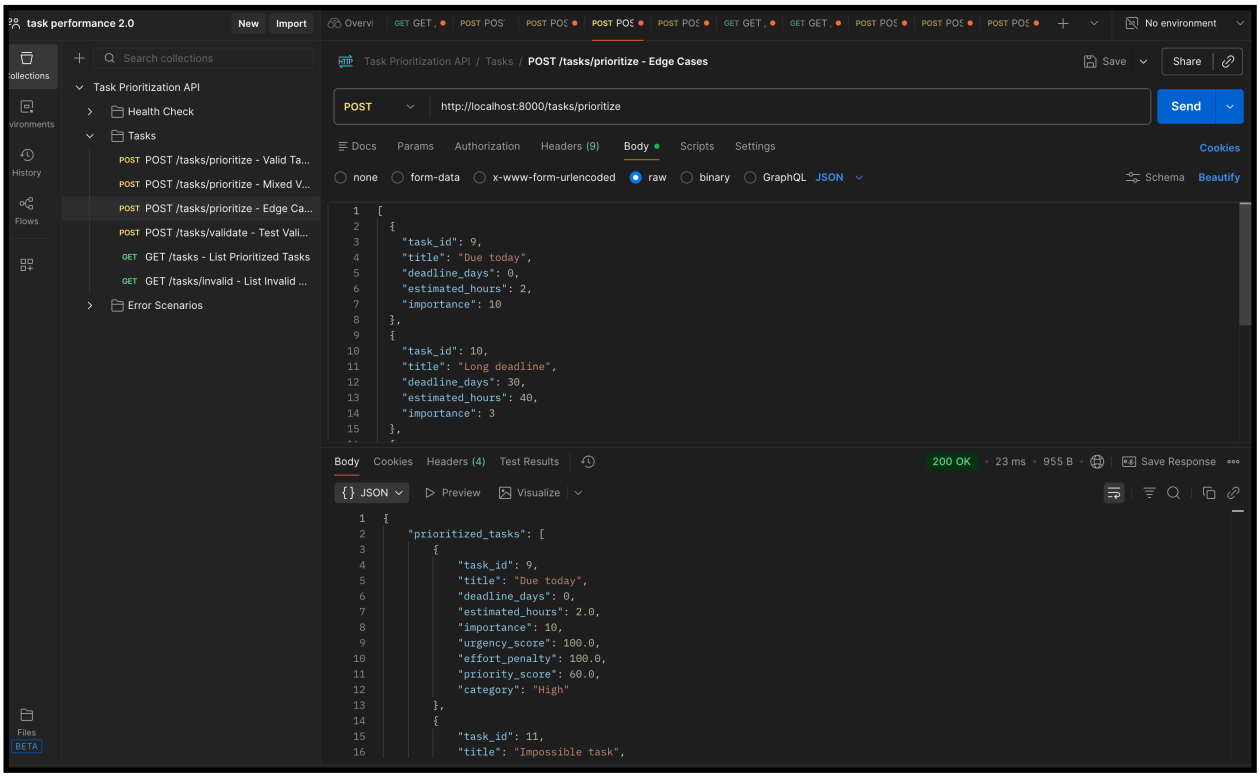
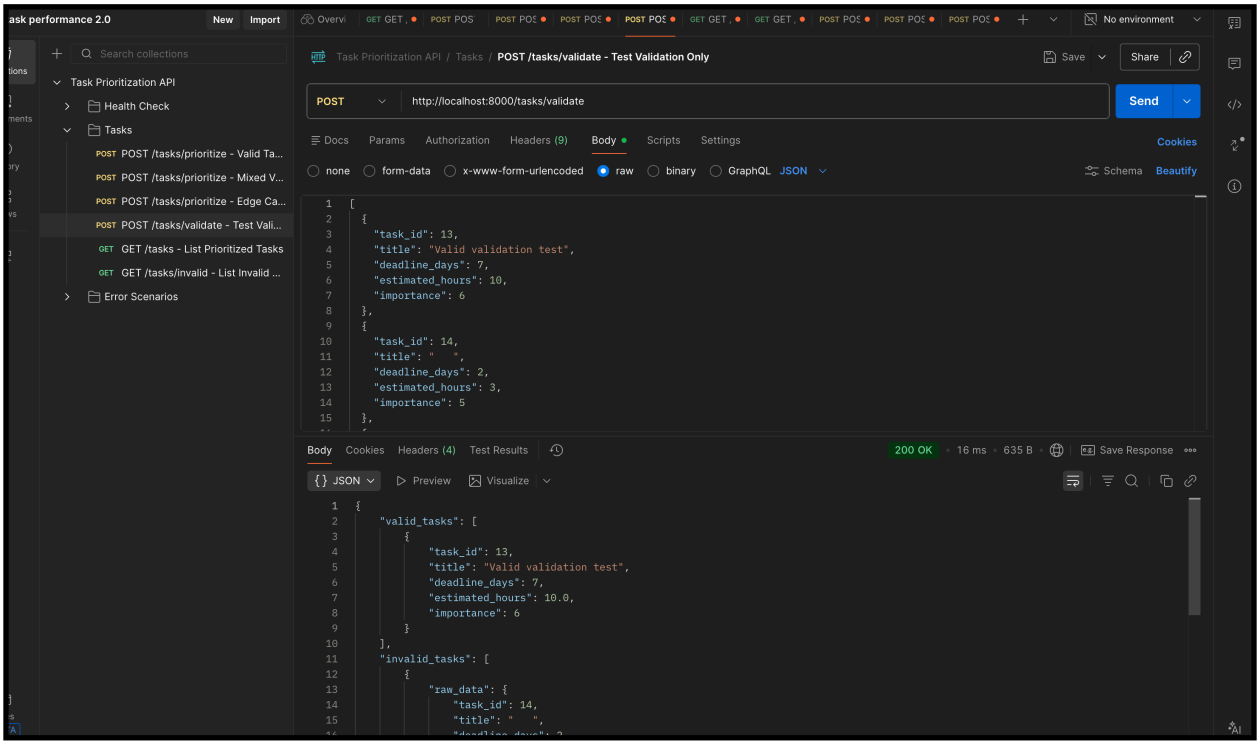
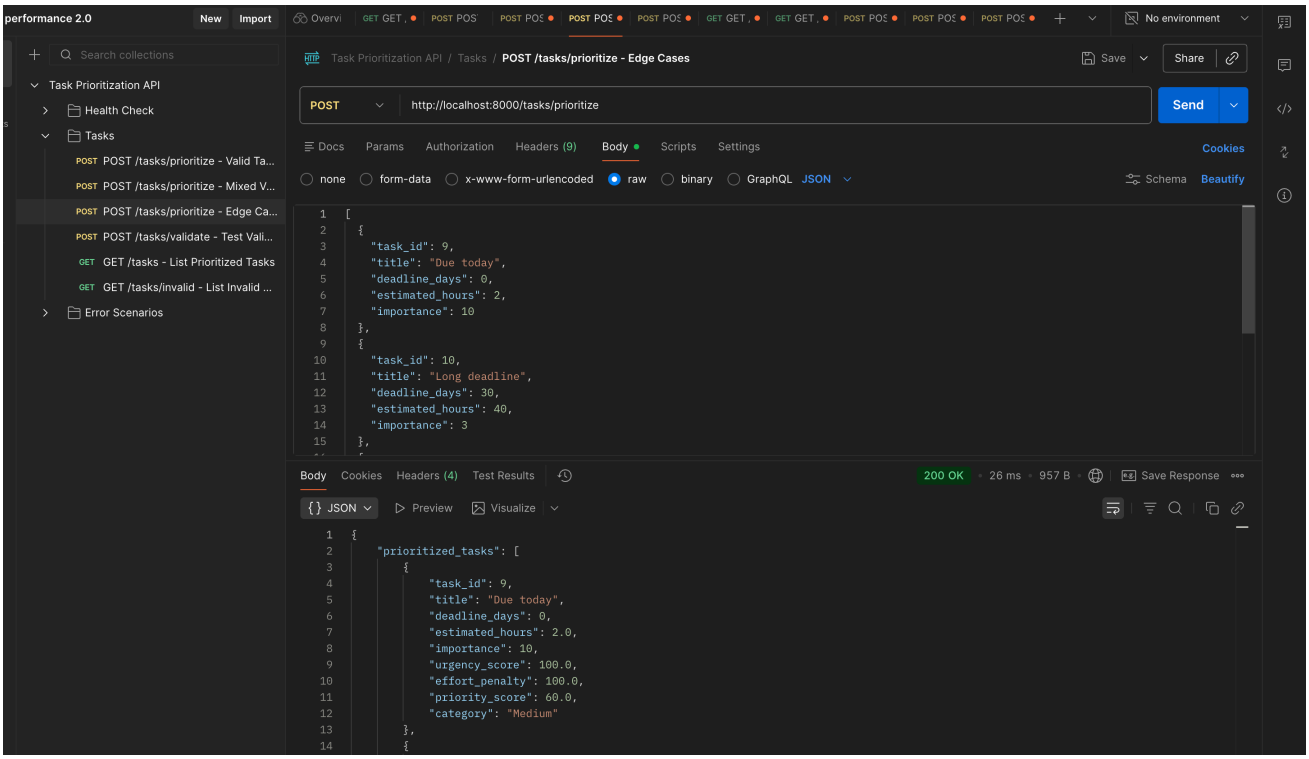
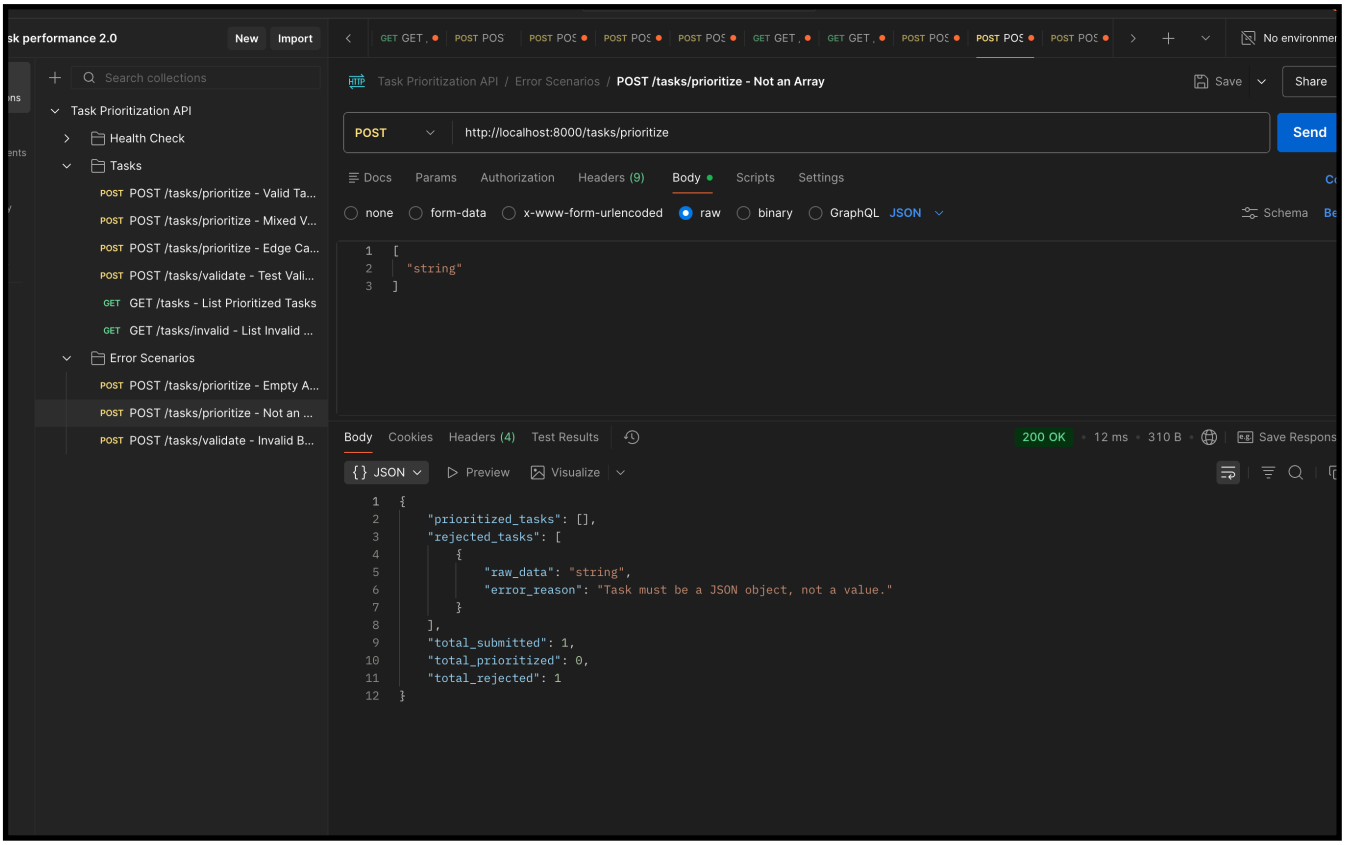

task performance 2.0
New
Import
Search collections
Task Prioritization API
Health Check
GET /health
Tasks
POST POST /tasks/prioritize - Valid Ta...
POST POST /tasks/prioritize - Mixed V...
POST POST /tasks/prioritize - Edge Ca...
POST POST /tasks/validate - Test Vali...
GET GET /tasks - List Prioritized Tasks
GET GET /tasks/invalid - List Invalid ...
Error Scenarios
POST POST /tasks/prioritize - Empty A...
POST POST /tasks/prioritize - Not an ...
POST POST /tasks/validate - Invalid B...
No environment
Task Prioritization API / Health Check / GET /health
Save
Share
GET
http://localhost:8000/health
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
1 [
2 "string"
3]
Body
Cookies
Headers (4)
Test Results
200 OK
36 ms
188 B
Save Response
{ } JSON
Preview
Visualize
1 {
2 "status": "ok",
3 "message": "Task Prioritization API is running."
4 }

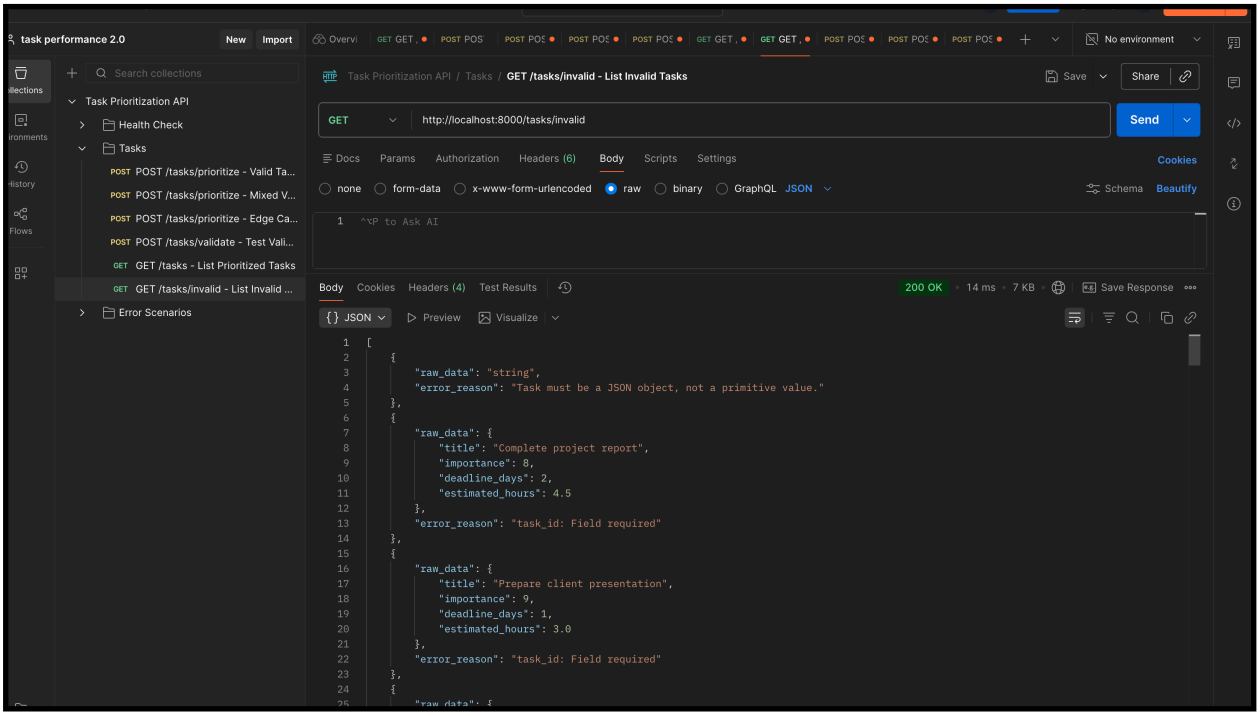

task performance 2.0
New
Import
Search collections
Task Prioritization API
Health Check
Tasks
POST POST /tasks/prioritize - Valid Ta...
POST POST /tasks/prioritize - Mixed V...
POST POST /tasks/prioritize - Edge Ca...
POST POST /tasks/validate - Test Vali...
GET GET /tasks - List Prioritized Tasks
GET GET /tasks/invalid - List Invalid ...
Error Scenarios
No environment
Task Prioritization API / Tasks / GET /tasks - List Prioritized Tasks
Save
Share
GET
http://localhost:8000/tasks
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
1 ^⌥P to Ask AI
Body
Cookies
Headers (4)
Test Results
200 OK
6 ms
7.1 KB
Save Response
{ } JSON
Preview
Visualize
1 [
2 {
3 "task_id": 1,
4 "title": "Complete project proposal",
5 "deadline_days": 2,
6 "estimated_hours": 4.5,
7 "importance": 9,
8 "urgency_score": 94.0,
9 "effort_penalty": 28.12,
10 "priority_score": 68.04,
11 "category": "Medium"
12 },
13 {
14 "task_id": 1,
15 "title": "Complete project proposal",
16 "deadline_days": 2,
17 "estimated_hours": 4.5,
18 "importance": 9,
19 "urgency_score": 94.0,
20 "effort_penalty": 28.12,
21 "priority_score": 68.04,
22 "category": "Medium"
23 },
24 {
25 "task_id": 1,
26 "title": "Complete project proposal",

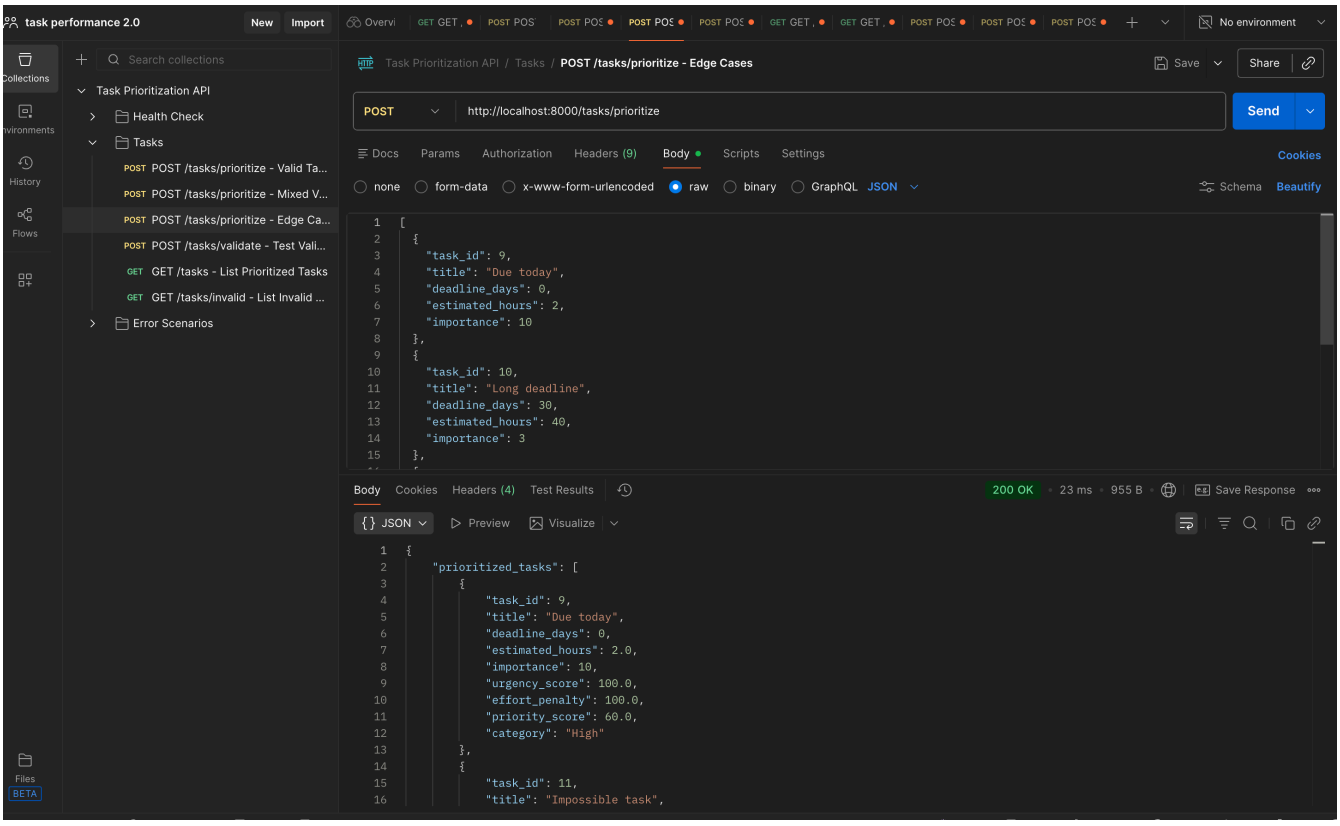

task performance 2.0
New
Import
Search collections
Task Prioritization API
Health Check
Tasks
POST POST /tasks/prioritize - Valid Ta...
POST POST /tasks/prioritize - Mixed V...
POST POST /tasks/prioritize - Edge Ca...
POST POST /tasks/validate - Test Vali...
GET GET /tasks - List Prioritized Tasks
GET GET /tasks/invalid - List Invalid ...
Error Scenarios
Task Prioritization API / Tasks / POST /tasks/prioritize - Edge Cases
Save
Share
POST
http://localhost:8000/tasks/prioritize
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
Body
Cookies
Headers (4)
Test Results
200 OK
23 ms
955 B
Save Response
JSON
Preview
Visualize
Files
BETA


ask performance 2.0
New
Import
Search collections
Task Prioritization API
Health Check
Tasks
POST POST /tasks/prioritize - Valid Ta...
POST POST /tasks/prioritize - Mixed V...
POST POST /tasks/prioritize - Edge Ca...
POST POST /tasks/validate - Test Vali...
GET GET /tasks - List Prioritized Tasks
GET GET /tasks/invalid - List Invalid ...
Error Scenarios
Task Prioritization API / Tasks / POST /tasks/validate - Test Validation Only
Save
Share
POST
http://localhost:8000/tasks/validate
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
Body
Cookies
Headers (4)
Test Results
200 OK
16 ms
635 B
Save Response
JSON
Preview
Visualize


performance 2.0
New
Import
Search collections
Task Prioritization API
Health Check
Tasks
POST POST /tasks/prioritize - Valid Ta...
POST POST /tasks/prioritize - Mixed V...
POST POST /tasks/prioritize - Edge Ca...
POST POST /tasks/validate - Test Vali...
GET GET /tasks - List Prioritized Tasks
GET GET /tasks/invalid - List Invalid ...
Error Scenarios
No environment
Task Prioritization API / Tasks / POST /tasks/prioritize - Edge Cases
Save
Share
POST
http://localhost:8000/tasks/prioritize
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
[
{
"task_id": 9,
"title": "Due today",
"deadline_days": 0,
"estimated_hours": 2,
"importance": 10
},
{
"task_id": 10,
"title": "Long deadline",
"deadline_days": 30,
"estimated_hours": 40,
"importance": 3
},
Body
Cookies
Headers (4)
Test Results
200 OK
26 ms
957 B
Save Response
{} JSON
Preview
Visualize
{
"prioritized_tasks": [
{
"task_id": 9,
"title": "Due today",
"deadline_days": 0,
"estimated_hours": 2.0,
"importance": 10,
"urgency_score": 100.0,
"effort_penalty": 100.0,
"priority_score": 60.0,
"category": "Medium"
},
{


sk performance 2.0
New
Import
Search collections
Task Prioritization API
Health Check
Tasks
POST POST /tasks/prioritize - Valid Ta...
POST POST /tasks/prioritize - Mixed V...
POST POST /tasks/prioritize - Edge Ca...
POST POST /tasks/validate - Test Vali...
GET GET /tasks - List Prioritized Tasks
GET GET /tasks/invalid - List Invalid ...
Error Scenarios
POST POST /tasks/prioritize - Empty A...
POST POST /tasks/prioritize - Not an ...
POST POST /tasks/validate - Invalid B...
No environmen
Task Prioritization API / Error Scenarios / POST /tasks/prioritize - Not an Array
Save
Share
POST
http://localhost:8000/tasks/prioritize
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
[
"string"
]
Body
Cookies
Headers (4)
Test Results
200 OK
12 ms
310 B
Save Respons
{} JSON
Preview
Visualize
{
"prioritized_tasks": [],
"rejected_tasks": [
{
"raw_data": "string",
"error_reason": "Task must be a JSON object, not a value."
}
],
"total_submitted": 1,
"total_prioritized": 0,
"total_rejected": 1
}


task performance 2.0
New
Import
Search collections
Task Prioritization API
Health Check
Tasks
POST POST /tasks/prioritize - Valid Ta...
POST POST /tasks/prioritize - Mixed V...
POST POST /tasks/prioritize - Edge Ca...
POST POST /tasks/validate - Test Vali...
GET GET /tasks - List Prioritized Tasks
GET GET /tasks/invalid - List Invalid ...
Error Scenarios
Task Prioritization API / Tasks / GET /tasks/invalid - List Invalid Tasks
Save
Share
GET
http://localhost:8000/tasks/invalid
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
^\P to Ask AI
Body
Cookies
Headers (4)
Test Results
200 OK
14 ms
7 KB
Save Response
JSON
Preview
Visualize
[
{
"raw_data": "string",
"error_reason": "Task must be a JSON object, not a primitive value."
},
{
"raw_data": {
"title": "Complete project report",
"importance": 8,
"deadline_days": 2,
"estimated_hours": 4.5
},
"error_reason": "task_id: Field required"
},
{
"raw_data": {
"title": "Prepare client presentation",
"importance": 9,
"deadline_days": 1,
"estimated_hours": 3.0
},
"error_reason": "task_id: Field required"
},


task performance 2.0
New
Import
Search collections
Task Prioritization API
Health Check
Tasks
POST POST /tasks/prioritize - Valid Ta...
POST POST /tasks/prioritize - Mixed V...
POST POST /tasks/prioritize - Edge Ca...
POST POST /tasks/validate - Test Vali...
GET GET /tasks - List Prioritized Tasks
GET GET /tasks/invalid - List Invalid ...
Error Scenarios
Task Prioritization API / Tasks / POST /tasks/prioritize - Edge Cases
Save
Share
POST
http://localhost:8000/tasks/prioritize
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
[
{
"task_id": 9,
"title": "Due today",
"deadline_days": 0,
"estimated_hours": 2,
"importance": 10
},
{
"task_id": 10,
"title": "Long deadline",
"deadline_days": 30,
"estimated_hours": 40,
"importance": 3
},
Body
Cookies
Headers (4)
Test Results
200 OK
23 ms
955 B
Save Response
JSON
Preview
Visualize
{
"prioritized_tasks": [
{
"task_id": 9,
"title": "Due today",
"deadline_days": 0,
"estimated_hours": 2.0,
"importance": 10,
"urgency_score": 100.0,
"effort_penalty": 100.0,
"priority_score": 60.0,
"category": "High"
},
{
"task_id": 11,
"title": "Impossible task",

Task Prioritization System

Technical Documentation

FastAPI + PostgreSQL

1. Overview

The Task Prioritization System is a robust, deterministic task prioritization engine built with FastAPI and PostgreSQL. It validates incoming tasks, calculates priority scores using a weighted formula, and categorizes them into High, Medium, or Low priority tiers.

The system is fully Dockerized, production-ready, and exposes a RESTful API for submitting, validating, and retrieving tasks.

2. Technology Stack

Technology	Role
Python	Core programming language
FastAPI	Web framework for building the REST API
PostgreSQL	Relational database for task persistence
Docker / Docker Compose	Containerization and orchestration
Pydantic	Data validation and schema enforcement
Git / GitHub	Version control and source hosting

3. Prioritization Logic

The system uses a mathematical model to ensure objective, deterministic task ranking. Every task is scored on a scale of 0 to 100.

3.1 Priority Formula

The core formula combines three weighted components:

```
priority_score = (0.50 × urgency_score) + (0.30 × importance_norm) −
(0.20 × effort_penalty)
```

All intermediate values are normalized to the range [0, 100] before blending.

3.2 Urgency Score

Urgency is derived from the number of days remaining until the task deadline:

```
urgency_score = clamp(100 - deadline_days × 3, 0, 100)
```

- deadline = 0 → urgency = 100 (task is due today or overdue)
- deadline = 33 → urgency = 1 (distant deadline)
- deadline > 33 → urgency = 0 (very distant deadline)

3.3 Importance Normalization

The user-supplied importance value (scale 1-10) is linearly mapped to [0, 100]:

```
importance_norm = ((importance - 1) / 9) × 100
```

3.4 Effort Penalty

The effort penalty compares estimated_hours against available working hours before the deadline (assuming 8 hours/day):

```
available_hours = deadline_days × 8
```

Calculation rules:

- If deadline_days == 0 and estimated_hours > 0: effort_penalty = 100
- Otherwise: effort_penalty = clamp((estimated_hours / available_hours) × 100, 0, 100)

A penalty of 100 means the task cannot realistically be completed before the deadline. It still appears in output but receives a heavy score reduction.

3.5 Priority Categories

Category	Score Range	Indicator
High	≥ 60	🔴 High
Medium	≥ 40 and < 60	🟡 Medium
Low	< 40	🟢 Low

3.6 Conflict Resolution

The weighted formula handles edge cases automatically:

- High importance + far deadline: importance boosts score, low urgency reduces it. Usually results in Medium priority.
- Low importance + near deadline: high urgency pushes towards Medium; cannot reach High without importance.
- Impossible task (hours > available): effort_penalty = 100, heavily reduces priority_score. Results in Low priority.

3.7 Deterministic Tie-Breaking

Tasks with equal priority_score are sorted by:

1. deadline_days ASC (sooner deadline comes first)
2. task_id ASC (lower ID comes first - stable and predictable)

4. Features

- Data Validation: Strict Pydantic schemas ensure task_id, importance (1-10), and estimated_hours are valid.
- Audit Logging: Successfully prioritized tasks are stored in the tasks table; failed validations are logged in invalid_tasks for debugging.
- Deterministic Sorting: Ties are broken by soonest deadline, then by Task ID.
- Dockerized: Ready for production deployment with docker-compose.

- Penalization Over Rejection: Impossible tasks are penalized, not removed, to maintain full system visibility.

5. API Endpoints

5.1 Task Endpoints

Method	Endpoint	Description
POST	/tasks/prioritize	Validate, score, and persist tasks to DB
POST	/tasks/validate	Dry-run validation (no DB storage)
GET	/tasks	Retrieve all prioritized tasks from DB
GET	/tasks/invalid	View audit log of rejected tasks

5.2 System Endpoints

Method	Endpoint	Description
GET	/health	Liveness check for the API
GET	/docs	Interactive Swagger UI

6. Getting Started

6.1 Prerequisites

- Docker and Docker Compose (for containerized setup)
- Python 3.9+ with pip (for local development)
- PostgreSQL (for local development without Docker)

6.2 Quick Start (Docker) - Recommended

3. **Clone the repository:**

```
git clone <repository-url>
```

4. **Run the application:**

```
docker-compose up --build
```

5. **Access the API:**

- API Base URL: http://localhost:8000
- Interactive Docs: http://localhost:8000/docs

6.3 Local Development (Python)

6. **Install dependencies:**

```
pip install -r requirements.txt
```

7. **Create a .env file with the following environment variables:**

```
DATABASE_URL=postgresql://postgres:password@localhost:5432/task_prioritization
```

8. **Run the development server:**

```
uvicorn main:app --reload
```

7. Environment Variables

Create a .env file in the project root with the following configuration:

```
DATABASE_URL=postgresql://postgres:password@localhost:5432/task_prioritization

POSTGRES_USER=postgres

POSTGRES_PASSWORD=password

POSTGRES_DB=task_prioritization

POSTGRES_HOST=localhost

POSTGRES_PORT=5432
```

8. Database Schema

8.1 Tasks Table

Stores all validated and successfully prioritized tasks. Key columns include:

- task_id - Unique identifier for the task
- title - Human-readable task name
- deadline_days - Days remaining until deadline
- estimated_hours - Estimated effort in hours
- importance - User-supplied importance (1-10)
- urgency_score - Computed urgency value

- effort_penalty - Computed effort penalty
- priority_score - Final computed priority score
- category - Assigned category: High, Medium, or Low

8.2 Invalid Tasks Table

Serves as an audit log for all failed task submissions. Key columns include:

- raw_data (JSONB) - The original submitted payload
- error_reason - Human-readable description of why validation failed

9. Example Request & Response

9.1 Request Payload (POST /tasks/prioritize)


```
[
  {
    "task_id": 1,
    "title": "Complete project proposal",
    "deadline_days": 2,
    "estimated_hours": 4.5,
    "importance": 9
  },
  {
    "task_id": 2,
    "title": "Review team pull requests",
    "deadline_days": 1,
    "estimated_hours": 2,
    "importance": 7
  },
  {
    "task_id": 3,
    "title": "Schedule quarterly meeting",
```

```
    "deadline_days": 5,
    "estimated_hours": 1,
    "importance": 5
  }
]
```

9.2 Response Output

```
{
  "prioritized_tasks": [
    {
      "task_id": 1, "title": "Complete project proposal",
      "urgency_score": 94.0, "effort_penalty": 28.12,
      "priority_score": 68.04, "category": "Medium"
    },
    {
      "task_id": 2, "title": "Review team pull requests",
      "urgency_score": 97.0, "effort_penalty": 25.0,
      "priority_score": 63.5, "category": "Medium"
    },
    {
      "task_id": 3, "title": "Schedule quarterly meeting",
      "urgency_score": 85.0, "effort_penalty": 2.5,
      "priority_score": 55.33, "category": "Medium"
    }
  ],
  "rejected_tasks": [],
  "total_submitted": 3,
  "total_prioritized": 3,
```

```
  "total_rejected": 0
}
```

10. Design Decision: Penalize vs. Reject

A key architectural decision in this system is to penalize tasks with impossible requirements rather than rejecting them outright.

When tasks have impossible requirements (e.g., estimated hours exceed available time before the deadline), the system applies an effort penalty to lower the priority score. This approach provides the following benefits:

- All submitted tasks remain visible in the system for complete transparency.
- Users can see how 'impossible' tasks are still ranked relative to feasible ones.
- The audit log captures why certain tasks receive lower priority scores.
- Data integrity is maintained: no silent data loss from rejection.

This aligns with the principle of maintaining full system visibility while still surfacing the most feasible, high-value tasks at the top of the priority list.

11. Project File Structure

```
TASK PERFORMANCE/
|
├── app/
|   ├── api/
```

```
|   |   └── routes.py
|   ├── db/
|   |   ├── database.py
|   |   └── models.py
|   ├── services/
|   |   ├── db_service.py
|   |   └── scoring_logic.py  ←  [MOST IMPORTANT]
|   ├── validators/
|   |   └── schemas.py
|   └── config.py
├── .env
├── docker-compose.yml
├── Dockerfile
├── main.py
├── README.md
└── requirements.txt
```

Note: scoring_logic.py is the most critical file in the project. It contains the full implementation of the priority formula, urgency score, importance normalization, effort penalty, tie-breaking logic, and categorization.

12. API Testing

A Postman collection is provided for testing all API endpoints. Import the file Task Prioritization API.postman_collection.json into Postman to get started.

The collection includes pre-configured requests for all endpoints, including sample payloads for prioritization and validation, and edge case scenarios (impossible tasks, invalid data).